



Gorgon: Accelerating Machine Learning from Relational Data

Matthew Vilim
Stanford

Alexander Rucker
Stanford

Yaqi Zhang
Stanford

Sophia Liu
Stanford

Kunle Olukotun
Stanford

Abstract—Accelerator deployment in data centers remains limited despite domain-specific architectures’ promise of higher performance. Rapidly-changing applications and high NRE cost make deploying fixed-function accelerators at scale untenable. More flexible than DSAs, FPGAs are gaining traction but remain hampered by cumbersome programming models, long synthesis times, and slow clocks. Coarse-grained reconfigurable architectures (CGRA) are a compelling alternative and offer efficiency while retaining programmability—by providing general-purpose hardware and communication patterns, a single CGRA targets multiple application domains.

One emerging application is in-database machine learning: a high-performance, low-friction interface for analytics on large databases. We co-locate database and machine learning processing in a unified reconfigurable data analytics accelerator, Gorgon, which flexibly shares resources between DB and ML without compromising performance or incurring excessive overheads in either domain. We distill and integrate database parallel patterns into an existing ML-focused CGRA, increasing area by less than 4% while outperforming a multicore software baseline by 1500×. We also explore the performance impact of unifying DB and ML in a single accelerator, showing up to 4× speedup over split accelerators.

Index Terms—database, machine learning, accelerator, CGRA, Plasticine, Gorgon

I. INTRODUCTION

Data centers are bifurcating: demand for high-performance, energy-efficient data analytics at scale is driving the development of new domain-specific architectures to supplement CPUs. Google and Microsoft have deployed ASICs to accelerate ML training and inference [1, 2], which are ubiquitous, massively parallel, and compute-intensive applications. Fixed-function designs exploit communication patterns specific to an application by tailoring a datapath and memory hierarchy; this approach avoids CPUs’ overheads at the cost of programmability.

Academia is not short of further accelerator proposals: database accelerators [3, 4], genomics accelerators [5], linear algebra accelerators [6], graph accelerators [7], deep learning accelerators [8, 9], and more. However, accelerators with limited programmability are untenable because data center applications are diverse and changing rapidly [10]. Data-center operators also

This material is based on research sponsored by DARPA under agreement #FA8750-17-2-0095 and by NSF under award #1937301. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright notation thereon. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of DARPA, NSF, or the U.S. Government. This research is also supported in part by a Herbert Kunzel Stanford Graduate Fellowship and members of the Stanford DAWN project: Teradata, Facebook, Google, Ant Financial, NEC, VMware, and Infosys.

```
SELECT
  KMEANS_INFER(model,
    [tA.c1, tA.c2, ..., tA.c7,
     tB.c2, tB.c3, ..., tB.c10]
  ) AS inf, SUM(tA.c8),
  SUM(tB.c11), SUM(tB.c12)
FROM tA INNER JOIN tB
ON tA.c0 = tB.c1
GROUP BY inf
ORDER BY inf
```

Fig. 1. This DB+ML query clusters and aggregates records.

favor homogeneous compute resources to ease management and avoid overly specialized hardware [10, 11]. So, despite incessant demand for higher performance, data centers have so far eschewed fixed-function accelerators beyond ML [10].

Unlike fixed-function accelerators, FPGAs have been widely deployed in data centers for diverse applications. Both Microsoft’s and Amazon’s public clouds provide FPGAs for search ranking [10], compression [12], bioinformatics [5], databases [13], DNNs [14], and more. Programmability drives FPGAs’ success, along with easily-reused primitives: SRAM scratchpads, arithmetic units (DSP), and DRAM memory controllers. Furthermore, FPGAs amortize silicon cost by sharing these components across applications and NRE cost by selling one chip to multiple data centers.

This flexibility is inefficient and impedes developer productivity, however. To support variable-width datapaths and arbitrary logic, FPGAs use combinational, bit-level interconnects that consume most of their area and power [15]. Furthermore, clock frequencies are limited by long combinational paths, and the combinatorial explosion in placement options slows placement and routing. For real-time applications—like serving database queries—FPGA toolchains are too slow to re-synthesize. Instead, FPGA overlay architectures provide rapidly-reconfigurable, but less efficient, coarse primitives [16].

Coarse-grained reconfigurable architectures (CGRA) are a promising alternative to both fixed-function accelerators and FPGAs, providing word-level reconfigurability with hardened functional units to improve performance without compromising programmability. Parallel patterns—map, filter, group-by, etc.—capture applications’ inherent parallelism without developer-imposing annotations. Spatial [17], a hardware-focused domain-specific language, efficiently compiles parallel patterns for reconfigurable accelerators.

We propose *Gorgon*, a unified data analytics CGRA for in-database machine learning (DB+ML) with ML-focused parallel patterns (map, reduce, etc.) and DB-focused parallel patterns (sort, join, group-by, etc.). Gorgon thus accelerates *full* data-

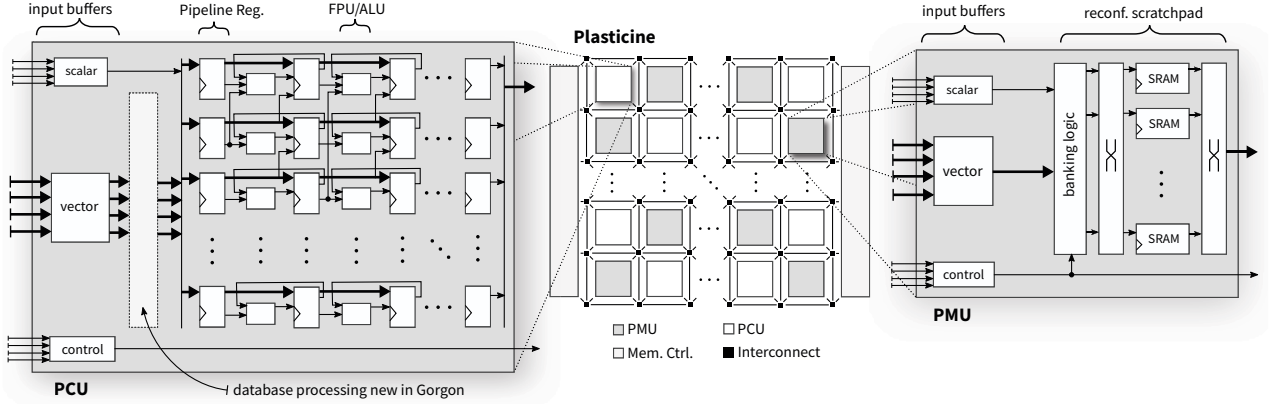


Fig. 2. Plasticine is a checkerboard layout of Pattern Compute Units (PCU) and Pattern Memory Units (PMU).

processing pipelines and is more reconfigurable than previous CGRAS. We believe these parallel patterns are both necessary and sufficient for a practical unified analytics accelerator. For example, online training of reinforcement learning models requires *joining* cause and effect data. Simultaneously, relational databases have been the gold standard for decades and still underpin new algorithms, including time series [18], spatial databases [19], graphs [20], and data programming [21].

Industrial DB+ML systems, like Google’s BigQuery ML and Apache’s MADlib, add ML inference and training to the relational model as new SQL operators (fig. 1). These DB+ML systems streamline data analytics, providing insights possible with neither DB nor ML alone. Databases analyze and aggregate structured data using handwritten queries but cannot learn from data; a SQL database can tally customers’ prior activity but cannot predict their future behavior. Conversely, ML models uncover the latent relationships between data, outperforming manually-derived heuristics, but cannot match DBs’ speed for structured processing (e.g., joins). DB and ML have been accelerated individually, but no single hardware platform accelerates both.

Gorgon adds database primitives to Plasticine [22], a CGRA for parallel patterns. These primitives share resources with each other and Plasticine’s existing microarchitecture, so they incur minimal performance and hardware overheads. Gorgon’s additions increase post-synthesis area by only 4%, and cycle-accurate simulation shows that Gorgon is 1500× faster and 1000× more energy efficient than a multicore software baseline. We also quantify the benefit of a shared HBM scratchpad and on-die links between DB and ML—both benefits of a multi-domain, high-performance CGRA.

The key contributions of this paper are:

- Detailed microarchitectural and system-level design of *low-overhead* database primitives embedded in a general-purpose reconfigurable accelerator.
- An evaluation of *end-to-end* system performance, including comparisons to software and independent DB and ML accelerators.

The remainder of this paper is organized as follows: §II

provides background on Plasticine and databases and §III discusses Gorgon’s design and microarchitecture. We evaluate the design in §IV and summarize related work in §V.

II. BACKGROUND

A. Plasticine & Spatial

Parallel patterns—map, reduce, filter, etc.—are powerful idioms for expressing data dependences and memory access patterns in parallel programs. These patterns let programmers express the parallelism found in many domains, including machine learning, graphs, and databases. Accelerators often require low-level programming that hampers developer productivity; parallel patterns simplify design by letting the compiler handle low-level details. Plasticine [22] is a spatially-reconfigurable CGRA designed to execute parallel patterns with 77× better performance per watt than FPGAs. Plasticine’s fabric (fig. 2) is a checkerboard grid of homogeneous compute and memory tiles connected by a reconfigurable interconnect. To account for process scaling, we use a larger 20×20 grid for Plasticine along with DDR4 memory.

The Pattern Compute Unit (PCU) executes the innermost pattern of a program, parallelizing across 16 SIMD lanes and pipelining across six stages. Each statically-configured stage executes a single arithmetic or logical operation, and pipeline registers between stages hold intermediate values and provide inter-lane communication. Vectorization and pipelining amortize the PCU’s control logic overhead and cost of the on-chip network.

The Pattern Memory Unit (PMU) is a reconfigurable scratchpad with 16 banks. Configurable addressing logic supports streaming FIFO accesses, strided accesses, and multi-buffering for coarse-grained pipelining. Each PMU has 256 KiB of single-cycle accessible SRAM, for a total of 50 MiB SRAM and 12.5 TB/s on-chip bandwidth.

A nested controller hierarchy sequences program execution and supports both fine-grained streaming and coarse-grained pipelining. A reconfigurable static-dynamic hybrid interconnect [23] connects PCUs and PMUs with 5.1 TB/s bisection bandwidth, and programmable state machines within the PCU

and PMU control the data flow across the chip. Plasticine’s memory controllers contain programmable address generators connecting the fabric to four channels of DDR4 DRAM.

Plasticine is programmed with Spatial [17], a language and compiler for programming accelerators with parallel patterns. A Plasticine-specific compiler backend [23] lowers Spatial into a distributed, streaming representation and parallelizes and pipelines loops. Parallelization includes both inner-loop vectorization across the 16 SIMD lanes and outer-loop parallelization across PCUS. Fine-grained pipelining keeps sequential stages in a PCU active processing data, and coarse-grained pipelining keeps sequential inner loops active throughout the chip. The compiler automatically banks and buffers memories to support these features.

B. In-Database Machine Learning

A vast amount of valuable business data is currently stored, un- or under-analyzed, in relational databases [24, 25]. The relational model [26] represents data as tables of records, where each record is a unique set of named fields. The field, or set of fields, that uniquely identifies each record within a table is called the *primary key*; a *foreign key* in one table points to a primary key in another, thus linking two tables. The Structured Query Language (SQL) provides a declarative interface for database queries [27], which abstracts away the underlying data representation.

Well-designed database schemas maintain relations in normal form, guaranteeing consistency by eliminating redundancy. To denormalize data for processing, databases use *joins*; a join between the primary key of one table and foreign key of another yields all rows where both keys match. SQL also allows data filtering based on a predicate (WHERE), sorting (ORDER BY), and group-based reductions (GROUP BY).

Relational databases are commonplace, but data analytics algorithms have moved beyond simple descriptive statistics (count, mean, max) to predictive ML models [28]. To understand correlations using SQL, one must describe their exact form (e.g., aggregating sales and profit by month). But, relationships in real-world data are sufficiently complex—in both the number of variables and their nonlinear correlations—that they cannot be identified by inspection; ML is needed to capture these underlying relationships. Just as SQL supports both online transactional and offline/nearline analytical processing, we support both inference and training. In production, models could be trained offline using a data analytics accelerator, migrated to the CPU, and used as part of transactional data processing. By using ML, data can be analyzed and acted upon more quickly—models can be retrained in seconds to hours, instead of days to months for an analyst to run queries and develop new business rules [29].

Current software for relational ML supports SQL operations extended with relatively shallow ML models. This software includes engines such as Apache’s Spark [28] and Google’s BigQuery ML [30] and in-database libraries such as Apache’s MADlib [31]. Most current ML research focuses on learning from unstructured data (e.g., text, images, and videos) using

Chip Area				Component Area	
Abs. (mm ²)		Rel. (%)		Abs. (μm ²)	Rel. (%)
PCU × 64	54.4	48	FPU/ALU	417 000	49
			Pipe. Reg.	144 000	17
			Data mov.	187 000	22
			Buffer	82 000	10
			Other	17 000	2
PMU × 64	33.9	30	Scratchpad	477 000	90
			Buffer	24 000	5
			Pipe. Reg.	23 000	4
			Other	6 000	1
Interconnect	18.8	17			
Memory Ctrl.	5.6	5			

Table 1. The original 16 × 8 Plasticine’s area in a 28 nm node [22].

massive deep neural nets [32]. In these problems, there is a complex relationship underlying the data (e.g., the language formed by a character sequence) that the model learns.

By contrast, much business data is already structured: records have fields with known datatypes and meanings, letting shallow models identify latent relationships. Because data is key to ML [1, 33], we emphasize bringing as much data as possible to a shallow model. Due to its low arithmetic intensity, this paradigm would overwhelm most deep learning accelerators’ i/o bandwidth.

III. DESIGN

Streaming dataflow accelerators targeting disjoint domains share common hardware, like memory; it makes little sense to build two accelerators when one accelerator’s hardware can be repurposed to target both domains. Consider Plasticine’s area breakdown in Table 1. Half the chip is composed of reconfigurable SRAM scratchpads, interconnect, and DRAM memory controllers; these are universal. A quarter is general-purpose hardware inside the PCU: data storage, data movement, buffers, and programmable control. Floating point units comprise the remaining quarter. If we consider these to be the ML-specific component, 75% of Plasticine is general-purpose and reusable. Gorgon repurposes these resources to target an application domain unsupported by Plasticine: databases.

However, DB and ML use different computation patterns. Machine learning algorithms are dominated by floating point stencil operations that are easily parallelized across SIMD lanes. By contrast, streaming DB operations have low arithmetic intensity but are difficult to vectorize because stream control is data-dependent. Prior database accelerators [3, 4] have avoided vectorization entirely.

A. Parallel Patterns for Databases

Gorgon extends Plasticine with a new set of parallel patterns: *partition*, *merge*, *group-by*, *join*, and *filter*. Each pattern is composable and scalable, using parallelization to trade off runtime with DRAM bandwidth and resource constraints. Each PCU can statically implement one pattern, and multiple PCUS

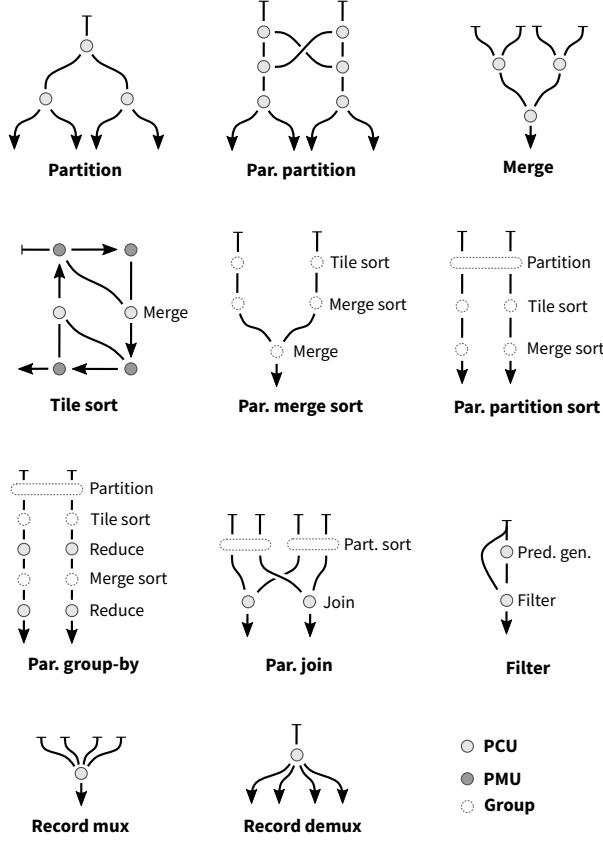


Fig. 3. Gorgon's new primitives. Gorgon can reconfigure its compute and memory tiles to trade off performance with resource requirements.

and PMUs can be arranged for higher performance. Figure 3 summarizes the PCU-PMU connectivity for each pattern.

Partition. Range partitioning is a preprocessing step that increases query parallelism by splitting a table into disjoint subtables on a key. To account for data skew, the key is sampled to identify ranges with roughly equal partition sizes. We implement range partitioning as a streaming tree of PCUs; records are compared against the partition boundary and move left or right down the tree.

Gorgon supports a reconfigurable number of partitions and fuses multiple trees to parallelize partitioning. Because per-stream activity drops by half at each level of the tree, we reduce the hardware needed by merging two half-active streams bound for the same partition. For example, fig. 3 depicts two streams partitioned four ways. The resources saved increase with the number of output partitions.

Merge. Sorting is both a SQL primitive (`ORDER BY`) and a preprocessing step before `JOINS` and `GROUP BYS`. Sorting is common, so parallelism and bandwidth efficiency are performance-critical. A merge consumes two sorted record streams and emits a single sorted stream. Repeated merges of smaller tables sort larger tables.

Parallel merge sorts bottleneck as parallelization increases.

A typical parallel implementation splits a table and sorts each subtable in parallel. Merging these sorted subtables yields the fully sorted table; however, this final merge cannot be parallelized, limiting the available speedup due to Amdahl's law. Instead, we use a partition-based sort, partitioning first and merge sorting subtables in parallel.

To reduce DRAM traffic, Gorgon tiles tables into SRAM scratchpads to sort without off-chip accesses and materializes sorted tiles to DRAM for further merges. Merge sort requires double-buffering with two independent streams from each buffer, so Gorgon requires at least four PMUs and two PCUs to sort a tile (fig. 3). We also increase the merge radix by building a tree of PCUs. Larger merge radices have no effect on the tree's aggregate throughput but reduce the number of reads and writes per record when merge sorting from DRAM. High-radix on-chip merge sorts and larger tile sizes are also possible by wiring together additional PMUs.

Group-By. A group-by collects records with shared keys into buckets and reduces records within each bucket. Because the number of buckets is variable and data-dependent, they cannot be kept on chip. Instead, pre-sorting records on their group-by key permits a one-pass streaming algorithm that reduces adjacent records with shared keys.

To parallelize group-by, we range-partition the table and sort tiles in scratchpads. Reducing tiles immediately before materialization to DRAM can significantly reduce memory traffic but still requires sorting the reduced tiles and a final reduction.

Join. A join on two record streams with a shared key concatenates records whose keys match into one larger record. Pre-sorting both tables on the join key allows a one-pass streaming algorithm. We partition tables on their join keys to allow joining multiple partitions in parallel.

Multi-way joins execute as a cascade of binary joins; because tables must be sorted on their join keys, each binary join must materialize before the next join can begin. To reduce DRAM traffic during multi-way joins, we tile and sort each binary join's output on-chip, reducing the number of off-chip merge iterations required to sort the table on the next join key.

Filter. To filter, the PCU consumes a record stream and a predicate stream indicating which records to omit. Typically, one or more PCUs compute the predicates and stream them to the PCU that filters the records. Filter parallelization is trivial (not shown).

Table Layout. Record processing often involves splitting or combining tables. Each PCU can multiplex or demultiplex a record stream by splitting or combining up to four streams. For example, if an ML model only infers on a subset of fields within a stream, Gorgon will demux records into two parallel streams, direct one through the ML model, and recombine them on the other side. To account for ML latency, we delay-balance these pipelines by inserting PMUs configured as FIFOs. Gorgon is a column store and merges columns to form tables when initially loading from memory; however, in this case, strided DRAM access avoids the need for a mux tree.

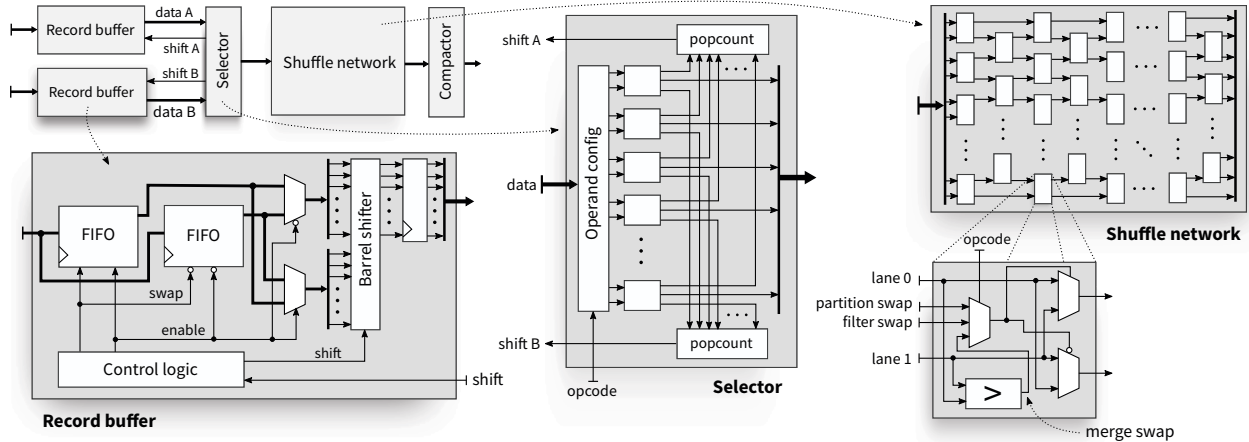


Fig. 4. Detail of Gorgon’s shared datapath for database operations within the PCU.

B. Microarchitecture

We propose low-overhead microarchitectural additions to the `pcu` to implement all primitives in a single reconfigurable vector datapath. Static configuration registers in each `pcu` select one of our five primitives: *partition*, *merge*, *join*, *group-by*, and *filter*. Designing a high-throughput vector pipeline for database operations is challenging: data dependences in streams create control dependences across vector lanes and stages. For example, when merging two sorted streams, the comparisons between leading records are dependent on the outcome of the previous merge.

Figure 4 summarizes the primary datapath components. Record vectors stream into record buffers, which allow a variable number of records (0–16) to be dequeued at a time. The selector determines how many records to dequeue from each record buffer and passes them to the shuffle network, which permutes lanes as necessary. Finally, the compactor accumulates records until all lanes are full.

Record Layout. The challenges of vectorized execution are compounded by the varied table layouts the datapath must accommodate. Common SQL patterns operate on multiple fields at a time, such as sorting on two keys. These *comparator keys* may span hundreds of bits; meeting timing at full throughput requires pipelined comparisons across multiple cycles. Our datapath deterministically resolves these control dependences without pipeline stalls, regardless of record schema.

Gorgon extends Plasticine to support records with arbitrary schemas. Configuration registers in each `PCU` set the input and output record streams’ schema: length, number of fields, and size of each field. Fields within a record may span one or more 32-bit words.

Figure 5 shows the two data layouts we considered for Gorgon. In the first layout, record fields span SIMD lanes in space, making each record visible at once to the control FSM and avoiding sequencing logic. However, this design does not scale: because keys’ size and position depend on the record layout, a wide (e.g., 16-lane) SIMD datapath requires a prohibitively

deep mux tree to reposition data. A preliminary in-space RTL design failed timing by over 700 ps.

Instead, we lay record fields out *in time*—each record stays in one vector lane, with its fields processed sequentially across pipeline stages, as shown in fig. 5. Fields wider than a single 32-bit lane are processed over multiple cycles. Execution is highly regular, even for long comparator keys, because each key comparison uses only data from a single vector lane. An added benefit of in-time records is that splitting and combining record streams is trivial: the PCU simply time-multiplexes its injection/ejection ports to direct fields between streams.

Although relational processing is well-suited to in-time records, ML models typically reduce across fields; on Gorgon, ML throughput is highest for in-space records. To maximize performance in both domains, we transpose record streams when interfacing DB and ML, writing the vector stream row-wise into an on-chip SRAM and reading out column-wise. Gorgon requires only a single PMU to transpose record streams between either layout at line rate, because each PMU contains 16 dual-ported SRAMs and operates as a double-buffer by atomically swapping base registers to pass a tile of data from producer to consumer. Figure 6 shows the banking scheme that readers and writers use to transpose vectors at full throughput [34].

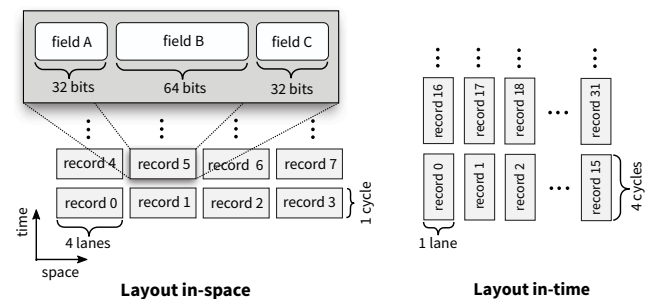


Fig. 5. The two SIMD record layouts we considered.

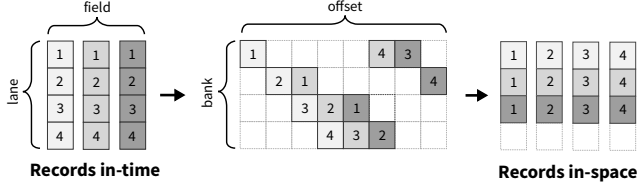


Fig. 6. The conflict-free banking scheme used to transpose records at line rate (only 4 lanes shown).

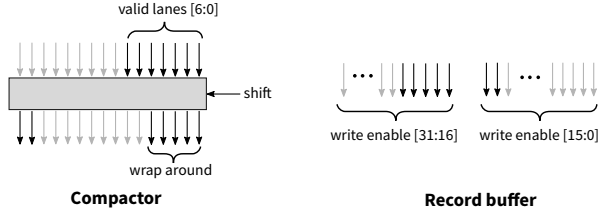


Fig. 7. A visual summary of the compactor working with the record buffer to concatenate partial vectors.

Record Buffers. Sequencing records in-time complicates the microarchitecture. Records may be compared multiple times if they are not selected during the first comparison, so we buffer each vector of records until all 16 have been processed. The record buffer can dequeue a variable number (0–16) of records while providing the next 16 records at its output. Merge, join, and filter produce one output stream from two input streams, so each PCU has two record buffers.

To fill all lanes with records, we use double-buffered memories within each record buffer, with read ports connected to a barrel shifter. This shifter provides a 16-record window into the 32 records stored in the memories, guaranteeing that records fill every lane even if some records have been dequeued. A counter in the control FSM tracks how many records have been processed and feeds the barrel shifter’s shift signal. When 16 records have been consumed, the double buffer swaps and presents the selector with 16 new records. The record buffer reuses existing input buffers within the PCU.

Selector. The selector (fig. 4) is the reconfigurable heart of the DB logic and determines how many records to dequeue from each record buffer. It contains per-lane comparators with a reconfigurable comparison function and operands. Using the comparators’ results, the selector chooses records from both streams and outputs a shift value (0–16) to each record buffer indicating how many records to dequeue.

Because we sequence records in time, comparisons between lanes are stateful and resolve over multiple cycles. An unresolved comparison simply indicates the keys are equal until that point, so there is no need to stall the pipeline. However, to avoid pipeline bubbles, comparison keys must be the first fields to arrive.

To *partition* a record stream, the selector compares all lanes against the partition boundary to determine their destination partitions. To *merge* two sorted streams, the selector compares one stream’s lanes with the other’s in reverse order [35],

choosing the 16 smallest of the 32 records compared. To *join* two sorted streams, the selector compares the first lane of the primary-key stream with all lanes of the foreign-key stream; it dequeues and concatenates matching records, or advances the smaller-valued stream if no records match. To *group* a sorted stream, the selector compares all lanes against the first lane, determining how many records share the reduction key. Group-by throughput is data-dependent and limited by the average group size. Sequential single-entry buckets on adjacent lanes are common, so a fast path detects this pattern and dequeues all records without reduction. To *filter*, the selector dequeues all records from one stream and a bitmask indicating the filter pattern from the other.

To meet timing, we retime the pipeline by registering the barrel shifter’s output in the record buffer, adding a single-cycle delay between the record buffer and selector. Because comparisons are only made using the heads of records, the shift signal from the selector (to the record buffer) remains steady after all lane comparisons resolve. Our pipeline typically has at least one spare cycle from non-key fields to accommodate the retiming delay. So, despite the cyclic control signal dependency, we do not incur a pipeline stall unless the entire record is used as the comparison key.

Shuffle Network. The right of fig. 4 shows the shuffle network, which swaps adjacent lanes based on a configurable function. As with the selector, comparisons in the shuffle logic resolve over multiple cycles; however, pipeline stalls are similarly unnecessary because data is shuffled in flight. To partition, the network swaps records bound for different output partitions until all records to each partition are adjacent. To merge, the network compares records on adjacent lanes and swaps until all lanes are sorted, forming an odd-even sorting network. Joined records pass through the shuffle network untouched, and grouped records use Gorgon’s reduction network to reduce fields across lanes. Since records are sorted on the group-by keys, we only need a single accumulator to store each bucket. To *filter*, the shuffle network interchanges invalid lanes with their neighbors until all valid lanes are adjacent.

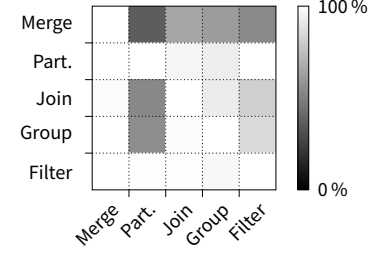
Compactor. Not every primitive fills all lanes with records (e.g., filter), so the compactor simplifies downstream processing by building full vectors from partial vectors. Concatenating partial vectors using an in-time record layout requires memory to buffer partial vectors during compaction. However, we avoid the overhead of a dedicated memory by relying on the record buffers of downstream PCUs. The compactor progressively rotates vectors using a barrel shifter, with an accumulator tracking how many lanes to shift. The downstream PCU gates each lane’s write enable signal in its record buffers for a partial enqueue. This scheme (fig. 7) avoids unnecessary complexity.

C. System Integration

ML Applications. Like DB operators, ML models use SIMD, pipeline, and stream-level parallelism. For inference, we parallelize ML models to match DB throughput where possible. Then, if resources permit, we replicate ML models and feed each

	PCU Area Overhead		PCU Power Overhead		Area Breakdown (%)		
	Abs. (μm^2)	Rel. (%)	Abs. (mW)	Rel. (%)	Comb.	Reg.	Mem.
Merge	23 081	7.0	17.5	6.4	67	33	0
Partition	8 394	2.6	10.5	3.9	24	76	0
Join	15 150	4.6	14.0	5.2	57	43	0
Group-by	14 688	4.5	13.6	5.0	53	43	4
Filter	12 437	3.8	13.0	4.8	49	51	0
All	24 024	7.3	26.6	9.8	66	32	2

a. PCU area and power overhead for primitives, individually and together.



b. Area correlation matrix.

Fig. 8. Area and power characterization of Gorgon’s new primitives.

with an independent DB stream for stream-level parallelism. For training, we accumulate a small batch of records (e.g., 64) in an SRAM. We pipeline and parallelize the forward and backward passes between records within each batch and perform mini-batch gradient descent after each batch.

Memory Access. Current datacenter I/O systems bottleneck accelerator performance by forcing the CPU to page buffers to and from the accelerator’s low-capacity memory during query execution [36]. OpenCAPI [36] is an emerging protocol that provides accelerators with high-bandwidth, coherent memory access. In an OpenCAPI-like environment, the CPU and accelerator share equal access to memory.

We believe that eventually most data will reside in accelerator memory. In future systems, the CPU will periodically—when new data arrives—update the relevant portion of accelerator memory. Therefore, Gorgon assumes low-overhead access to a large pool of DDR4 memory—either the CPU’s host memory through a fast, coherent interface or the accelerator’s own memory pool. Gorgon queries tables on the CPU’s behalf and returns a reduced view of the data—as a learned model or summarized statistics.

IV. EVALUATION

We evaluate Gorgon on a wide range of DB+ML workloads with cycle-accurate simulation, quantifying area overhead, the degree of reuse between Gorgon’s primitives, and performance.

A. Area Overhead

We implement Gorgon’s microarchitectural additions with Chisel [37] and verify them with handwritten test benches. We synthesize with Synopsys Design Compiler [38] and a 15 nm predictive PDK [39]. Because the 15 nm PDK does not have an SRAM compiler, we use latch-based arrays for small memories and scale large memories from the industrial 28 nm PDK used for Plasticine [22]. Gorgon meets timing at 1 GHz, with the critical path starting from the record buffer’s retiming register, through the selector’s popcount logic, and ending at the record buffer’s control logic.

To demonstrate Gorgon’s inter-primitive hardware reuse, we synthesize design variants with each primitive alone and all primitives combined. Figure 8a shows the absolute and PCU-relative area and power increases of each design point. The

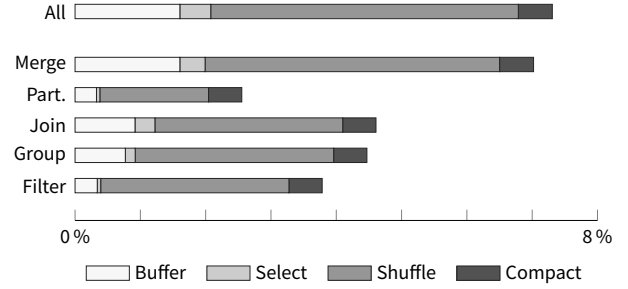


Fig. 9. Area overhead incurred by each component and all components together normalized to PCU area.

total area of independently-synthesized primitives far exceeds their combination; we achieve high reuse by designing one datapath with per-primitive reconfigurable operands. Merge is the largest operation (7% overhead) but contains hardware needed for the other primitives, making them almost free.

The database additions add less than 8% area to each PCU, which corresponds to less than 4% added chip area. Figure 9 shows the component area breakdown of the design synthesized with each primitive alone and all primitives together, normalized to a single PCU. The area increase is dominated by the shuffle network, which forms roughly half the database logic. Savings across primitives come from sharing input buffers, registers, comparators, and multiplexers in the datapath. Figure 8b shows the area reuse correlation between each primitive; the matrix shows a measure of the hardware shared between pairs of operations in the datapath. Adding additional primitives to merge is low-overhead, but the inverse is not necessarily true.

B. Microbenchmarks

Gorgon’s primitives permit flexible and reconfigurable parallel patterns that trade off runtime against on-chip resources and DRAM bandwidth (fig. 10). When Gorgon accesses high-bandwidth, low-capacity memories like HBM, its vector datapath and stream-level parallelism easily saturate the available bandwidth. However, given DRAM bandwidth constraints, Gorgon reconfigures to prioritize bandwidth efficiency.

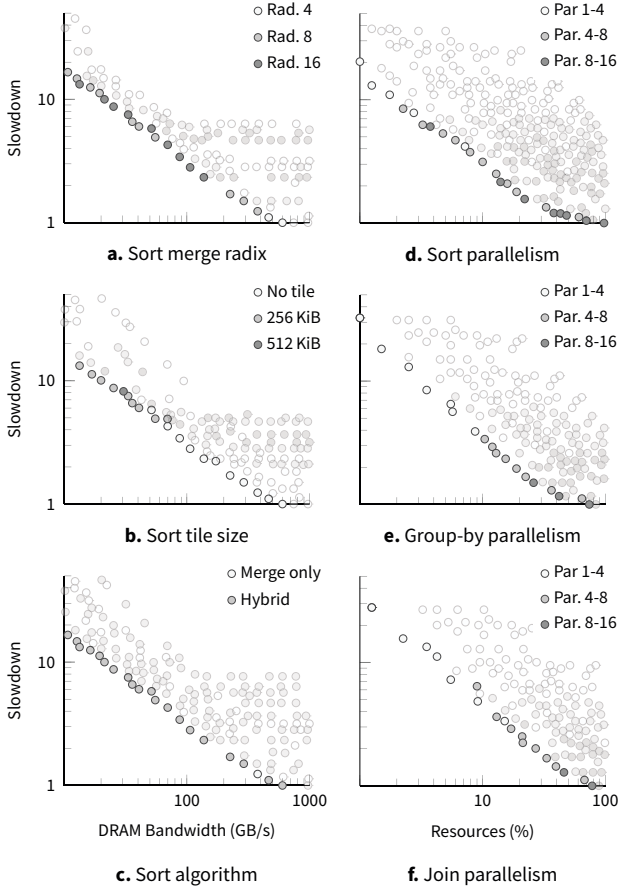


Fig. 10. Trade-off between DRAM bandwidth and runtime at fixed 10% resource constraint (a, b, c), and trade-off between resource use and runtime at fixed 1 TB/s DRAM bandwidth (d, e, f). For legibility, we plot only a subset of valid design points.

To show Gorgon’s flexibility, we simulate a set of microbenchmarks to find the Pareto frontier given a set of constraints. For example, fig. 10abc show how the Pareto-optimal sorting design changes as DRAM bandwidth scales. At low bandwidths, the best configurations use large merge radices and scratchpads, which both reduce the number of required DRAM accesses per record to sort the table. Gorgon’s hybrid partition-based sort outperforms a merge-only sort at nearly any bandwidth. Similarly, fig. 10def show that at a fixed 1 TB/s memory bandwidth, Gorgon’s primitives’ performance scales with increasing stream-level parallelism. However, performance tapers off as Gorgon becomes bandwidth-limited.

C. Simulation

We evaluate Gorgon’s end-to-end performance with a custom cycle-accurate simulator written in C++; our simulator integrates Ramulator [40] for DDR4 and HBM simulation. Gorgon lacks a query compiler, so we manually plan each query and program the DB portion by hand. We write the ML portion in

Software	DDR4	4 channels at 85 GB/s
	HBM	none
Hardware	Hardware	Intel Xeon E7-8890 v3 at 2.5 GHz
	DDR4	4 channels at 100 GB/s
Gorgon	HBM	16 channels (two stacks) at 256 GB/s
	Hardware	200 PCUS & 200 PMUS
	DB+ML link	5.1TB/s network bisection bandwidth
	No HBM	Derived from HBM
No DDR4	Derived from DRAM	Gorgon none
	Split 32 GB/s	Derived from DB+ML link
Split 16 GB/s	Derived from DB+ML link	Gorgon 31.5 GB/s, 400 ns lat. (PCIe 4.0 x16)
		15.8 GB/s, 400 ns lat. (PCIe 4.0 x8)

Table 3. A summary of evaluated design points; for brevity, only differences are shown where possible.

Spatial, combine the DB and ML dataflow graphs, and pass them to a custom place and route tool. The entire DB+ML application is placed and routed on the accelerator fabric to account for the on-chip interconnect’s latency and bandwidth.

Benchmarks. There is no accepted benchmark set for hybrid DB+ML workloads, so we use the queries shown in fig. 11. These queries stress a variety of DB and ML behaviors, including multi-way joins and multi-model inference and training. For example, Q5 uses one predicted value as an input to another model, and Q9 uses inference to optimize the training of a more complex model. We control the data skew to ensure realistic filter behavior; table 2 shows the input and output tables’ size and the number of records evaluated by the ML models.

Mapping to Gorgon. Figure 13 shows the breakdown between DB and ML resources when mapping applications to Gorgon. To account for technology scaling, we use a larger grid size than Plasticine: 200 PCUS and 200 PMUS (20 × 20). The ratio between memory and compute varies, especially for the ML portion; for example, Q4’s ML uses much more memory than compute, while Q1 is the opposite. The MLP training in Q9 uses almost all on-chip memory to materialize intermediate results and compute the backward gradient. This imbalance motivates a converged accelerator—one that can flexibly allocate resources between DB and ML. Split accelerators, such as an FPGA for DB and a GPU for ML, would each have to provision for the worst case, leading to poor average resource usage.

Comparison with Apache MADlib. No existing hardware platform accelerates DB+ML workloads, so we compare against a multicore software baseline. We benchmark against Apache MADlib on an Intel Xeon E7-8890 v3 with PostgreSQL parallelization enabled with data sizes from table 2; results are shown in fig. 12.

Speedups range from 300× to 6000× and energy efficiency (estimated by multiplying runtime with design power) improvements range from 200× to 3000× relative to MADlib. Our

Query 1

```
SELECT
  SVM_INFER(model,
    [tA.c1, ..., tA.c16]
  ) AS inf, tA.c17, tA.c18,
  tA.c19, tA.c20
FROM tA
WHERE inf > 0.5
ORDER BY tA.c0 DESC
LIMIT 10
```

Query 2

```
SELECT
  KMEANS_INFER(model,
    [tA.c1, ..., tA.c7,
     tB.c2, ..., tB.c10]
  ) AS inf, SUM(tA.c8),
  SUM(tB.c11), SUM(tB.c12)
FROM tA INNER JOIN tB
  ON tA.c0 = tB.c1
GROUP BY inf
ORDER BY inf
```

Query 3

```
SELECT
  MLP_INFER(model,
    [tA.c6, ..., tA.c21]
  ) AS inf, tA.c0
FROM tA
  WHERE tA.c3 < tA.c1 < tA.c2
  AND tA.c4 != tA.c5
  AND inf > 1000
ORDER BY inf DESC
```

Query 4

```
SELECT LINREG_INFER(model,
  [tA.c1, ..., tA.c8,
   tC.c2, ..., tC.c5]
) AS inf, tC.c1,
FROM (SELECT SUM(tB.c2),
  SUM(tB.c3), tB.c1,
  SUM(tB.c4), SUM(tB.c5),
  FROM tB GROUP BY tB.c1
) AS tC
INNER JOIN tA ON tC.c1 = tA.c0
ORDER BY inf, tC.c1 DESC LIMIT 10
```

Query 5

```
SELECT LINREG_INFER(modelA,
  [tB.c2, ..., tB.c13]
) AS infA,
LOGREG_INFER(modelB,
  [tA.c1, ..., tA.c5, infA,
   tC.c1, ..., tC.c5]
) AS infB, tA.c0, tC.c0
FROM tA INNER JOIN tB
  INNER JOIN tA AS tC
  ON tA.c0=tB.c0 AND tC.c0=tB.c1
WHERE infB > 0.5
ORDER BY tA.c0, tC.c0
```

Query 6

```
SELECT
  COUNT(*), AVG(tp1),
  AVG(tp2), tA.c0
FROM (SELECT
  LOGREG_INFER(model,
    [tA.c1, ..., tA.c16]
  ) AS inf,
  inf && !tA.c17 ? 1 : 0 AS tp1,
  !inf && tA.c17 ? 1 : 0 AS tp2,
  FROM tA
) AS tB
GROUP BY tA.c0 ORDER BY tA.c0
```

Query 7

```
SELECT LINREG_TRAIN(tC.c5,
  [tA.c1, ..., tA.c13,
   tC.c2, tC.c3, tC.c4]
) AS model,
FROM (SELECT SUM(tB.c2),
  SUM(tB.c3), tB.c1,
  SUM(tB.c4), SUM(tB.c5))
  FROM tB GROUP BY tB.c1
) AS tC
INNER JOIN tA ON tC.c1 = tA.c0
WHERE tC.c5 > 0
```

Query 8

```
SELECT
  LOGREG_TRAIN(tA.c2,
    [tA.c3, ..., tA.c10]
  ) AS model
FROM tA
WHERE tA.c2 != 0
```

Query 9

```
SELECT MLP_TRAIN(tB.c0,
  [tB.c1, ..., tB.c15]
) AS mlp_model
FROM (
  SELECT tA.c0, ..., tA.c16,
  LINREG_INFER(linreg_model,
    [tA.c1, ..., tA.c16]
  ) AS inf FROM tA
  WHERE inf/tA.c0 > 1.1
  OR inf/tA.c0 < 0.9
) AS tB
```

Fig. 11. The DB+ML queries we use to evaluate Gorgon.

	Table Rows				Table Columns				ML Count		Description
	A	B	C	Out	A	B	C	Out	Inf.	Tr.	
Q1	1M	-	-	100k	21	-	-	5	1M	-	Score each row, and return the top 10 scores.
Q2	1M	10M	-	16	9	13	-	4	10M	-	Categorize records and describe each category.
Q3	1M	-	-	45k	22	-	-	2	1M	-	Evaluate an MLP for a filtered set of records.
Q4	1M	10M	1M	10	9	6	6	2	717k	-	Aggregate B and join A-B. Return the top joined rows.
Q5	100k	1M	100k	861k	6	14	6	3	2M	-	Score edges (B) and classify node pairs (A) with scores.
Q6	1M	-	-	8	18	-	-	4	1M	-	Determine error rates for sub-populations.
Q7	1M	10M	1M	-	14	6	6	-	-	717k	The training equivalent of Q4, trained only on labeled data.
Q8	1M	-	-	-	11	-	-	-	-	500k	Train a classifier only on labeled rows.
Q9	100k	-	-	-	17	-	-	-	100k	69k	Train an MLP on rows with large errors on a simple model.

Table 2. Explanations of our benchmark queries with record lengths, input and output record counts for DB, and input record counts for ML.

	Runtime		Energy	
	Gorgon (ms)	MADlib (s)	Gorgon (J)	MADlib (kJ)
Q1	1.2	3.5	0.3	0.6
Q2	24.9	144.9	7.9	23.9
Q3	5.4	5.6	1.3	0.9
Q4	6.1	8.7	1.2	1.4
Q5	32.0	52.4	7.1	8.6
Q6	0.6	2.9	0.2	0.5
Q7	46.3	18.9	12.8	3.1
Q8	26.6	9.6	1.5	1.6
Q9	7.0	10.2	1.6	1.7

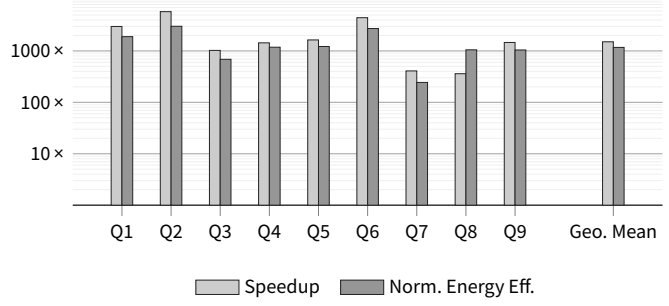


Fig. 12. Speed and energy efficiency improvement compared to Apache MADlib.

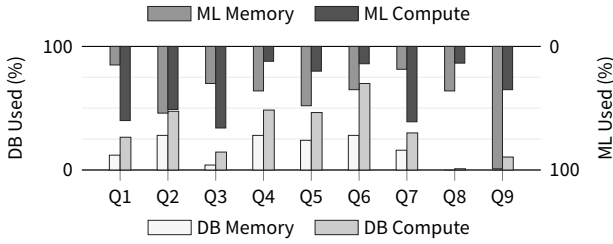


Fig. 13. The percentage of Gorgon’s memory and compute resources used by DB (from bottom) and ML (from top) processing.

average speedup is 1500 $\times$, and our average energy efficiency improvement is 1000 $\times$. Speedups are roughly 3 $\times$ higher for inference-only (Q1–Q6) than for training queries (Q7–Q9) because inference can be fully pipelined. By contrast, training requires reduction within each batch to compute the average gradient; weight updates create a loop-carried dependency that cannot be fully pipelined, limiting the achievable FLOPS. Nonetheless, Gorgon provides an average of 750 $\times$ speedup on the training applications.

Split Accelerators. One advantage of a unified data analytics platform is higher bandwidth between application domains. Typical accelerators communicate with each other and the host over PCIe; an accelerator might have a local HBM scratchpad, but the data would reside in a high-capacity remote memory. A unified accelerator alleviates the communication bottleneck.

We simulate the impact of this bottleneck by imposing a bandwidth constraint on all on-chip links between DB and ML processing. Most of our queries have multiple independent streams on-chip connecting DB+ML; we restrict the cumulative bandwidth and latency of these links to model PCIe 4.0 x16 and x8. Although we evaluate the split accelerator scenario using Gorgon, these bandwidth limits would apply to *any* split DB+ML system.

The impact of limited bandwidth on performance depends heavily on the bandwidth between DB and ML. Training applications are typically compute-bound, so there is a minimal (under 5%) performance impact from splitting training and DB, as shown in fig. 14. However, inference has higher throughput

and therefore suffers performance losses of almost 80% on Q1 and Q6 and a mean slowdown of almost 60%.

High Bandwidth Memory. A general-purpose accelerator better justifies HBM’s expense. Gorgon’s highly-parallel stream processing benefits greatly from HBM’s increased bandwidth, which is easily re-purposed across applications. We perform two sensitivity studies, shown in fig. 14, to quantify the impact of HBM on Gorgon’s performance. The first study explores removing HBM from Gorgon. For compute-intensive training queries, we do not see a significant difference, but inference queries have a mean slowdown of over 20% and a maximum slowdown of over 50%. Removing HBM decreases Gorgon’s sorting performance by forcing more DDR4 accesses.

We also evaluate the impact of removing DDR4 DRAM from the system and storing all data in HBM; however, this design is less feasible due to the cost and capacity limit (approx. 16 GiB) of HBM. Q5 has a 3.3 $\times$ speedup for this configuration, because it is heavily bottlenecked on the input stream and does not sort data. Using HBM exclusively gives a 1.3 $\times$ overall speedup.

Stream-Level Parallelism. To evaluate the impact of stream-level parallelism (fig. 15), we individually disable ML and sort parallelization. In some queries, sorted data streams directly into the ML models, so stream-level parallelism is the same. Q8 has neither parallel sort nor parallel ML in the optimized case, so its runtime does not change.

Disabling stream-level parallelization results in major performance losses. Q2 and Q4 pre-sort large tables and thus are slowed down by 75%; disabled ML parallelization impacts these queries less. Other queries, like Q3, Q5, and Q6, have closely-linked sorts and inference and have a significant performance impact from turning either off. Finally, most training queries are ML-limited and do not benefit from DB, except Q7 due to its group-by preprocessing.

V. RELATED WORK

We summarize relevant work in database acceleration and in-database ML and describe the differences from Gorgon.

Database Acceleration ASICs. Q100 is a database accelerator composed of fixed-function ASIC tiles for each database primitive [3]. These tiles are connected by a NoC, and a

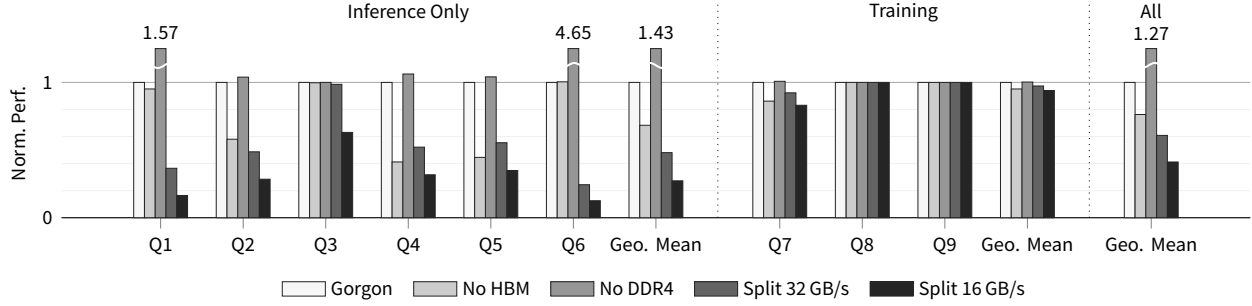


Fig. 14. The performance of alternate Gorgon hardware configurations, normalized to Gorgon’s baseline configuration. Configurations are summarized in table 3.

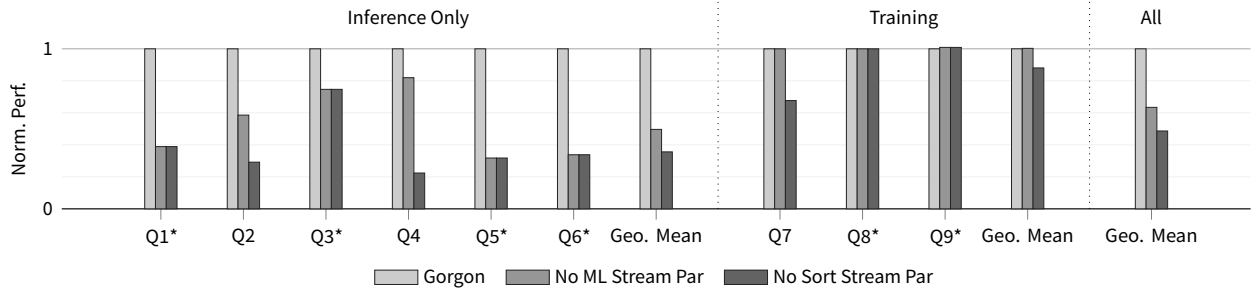


Fig. 15. Gorgon’s slowdown when ML or sort stream-level parallelization are turned off. Queries with linked ML and sort parallelization are indicated with an asterisk (*).

planner schedules queries onto the tiles spatially and temporally. Q100 achieves an average 10× performance improvement over software with 15% the area and energy of a reference Xeon CPU. A follow-up study compares a systolic array variant of Q100 with the fixed-function original [4].

Q100 is *not* a vector architecture (instead relying on stream-level parallelism) and was evaluated with 40 GB/s DRAM bandwidth, but HBM-enabled FPGAs providing 460 GB/s are already commercially available [41]. Gorgon scales to wide SIMD widths and conserves DRAM bandwidth when needed.

Gorgon is embedded into a flexible compute fabric while Q100 only accelerates databases. Besides sharing hardware between Gorgon primitives, we reuse most of Plasticine’s existing hardware, including the interconnect, PCU resources, and, most importantly, the on-chip SRAM. By building one large, shared accelerator for all applications, more of these supporting resources can be provided: a database-only accelerator with large SRAM scratchpads and multiple HBM stacks would be hard to justify economically.

FPGA Database Accelerators. FPGAs are widely deployed to accelerate transactional and analytical database processing [42]. Prior work with FPGAs accelerates either operators [35, 43, 44] or full queries [45]. FPGAs are widely deployed in data centers [10] but provide lower performance per watt than CGRAS [22]. FPGAs are flexible and provide a sea of logic primitives, registers, and SRAM embedded in a programmable interconnect. However, their development cycle is slow and typically involves programming with low-level RTL languages.

Furthermore, FPGAs cannot capitalize on their reconfigurability because their toolchain is too slow to reconfigure them in real time. Instead, FPGAs often employ overlay architectures [16] that can reconfigure to serve incoming queries.

Domain-Agnostic Dataflow Accelerators. Other dataflow accelerators target multiple domains. The design proposed by [46] is a mesh of sparse processing units (SPU) connected through a NoC. Each SPU contains a programmable systolic array with hardware support for on-chip sparse memory accesses, providing speedups for ML, graph, and database workloads. On-chip sparsity supports hash join, but only if the entire table fits in the limited scratchpad. The database speedups are similar to Q100, averaging 10× over a software baseline.

In-Database Machine Learning. DANA [47] accelerates ML on an FPGA connected to a PostgreSQL database running in software, providing a 4× speedup over MADlib by bypassing the CPU and allowing the FPGA to read directly from the database buffer pools. Gorgon accelerates *both* domains, avoiding the same CPU overheads.

A variety of software solutions integrate ML with database operations, including Apache’s Spark [28] and MADlib [31], Greenplum Database [48], and Google’s BigQuery ML. Spark is an open-source distributed data-processing engine for various workloads, including SQL, machine learning, and graph processing. BigQuery ML can run ML models in its cloud data warehouse [30]. MADlib is an open-source project that provides SQL-based ML algorithms that run in a database.

VI. CONCLUSION

A unified, performant, and flexible accelerator—the ideal, combining the advantages of fixed-function accelerators and FPGAs—would simplify data center management by keeping compute resources homogeneous. Furthermore, accelerator unification enables new, cross-domain applications like DB+ML. Gorgon demonstrates the potential for reuse between accelerators by embedding database-inspired parallel patterns (with 4% area overhead) in a general-purpose fabric originally designed for ML; it is also the first system to fuse and accelerate DB+ML processing on a single chip, accelerating *full* data-processing pipelines. We demonstrate a 1500× speedup over Apache MADlib.

Gorgon is better-suited to data center integration than a sea of heterogeneous accelerators and faster too: we demonstrate up to a 4× speedup by unifying disparate domains in a single chip with a high-bandwidth interconnect. Furthermore, a unified platform shares expensive resources like HBM, on-chip scratchpads, and large dice in advanced nodes. Gorgon’s stream-level parallelism and vector datapath easily saturate an HBM scratchpad’s bandwidth; when accessing high-capacity, lower-bandwidth memories, Gorgon reconfigures to become DRAM-bandwidth-efficient. By adding database support to an ML-focused accelerator, Gorgon enables more powerful ML pipelines than existing accelerators.

REFERENCES

- [1] N. P. Jouppi *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, ser. ISCA ’17. New York, NY, USA: ACM, 2017, pp. 1–12.
- [2] “Graphcore,” <https://www.graphcore.ai/>.
- [3] L. Wu, A. Lottarini, T. K. Paine, M. A. Kim, and K. A. Ross, “Q100: The architecture and design of a database processing unit,” *SIGARCH Comput. Archit. News*, vol. 42, no. 1, pp. 255–268, Feb. 2014.
- [4] A. Lottarini, J. a. P. Cerqueira, T. J. Repetti, S. A. Edwards, K. A. Ross, M. Seok, and M. A. Kim, “Master of none acceleration: A comparison of accelerator architectures for analytical query processing,” in *Proceedings of the 46th International Symposium on Computer Architecture*, ser. ISCA ’19, New York, NY, USA, 2019.
- [5] Y. Turakhia, G. Bejerano, and W. J. Dally, “Darwin: A genomics co-processor provides up to 15,000× acceleration on long read assembly,” *SIGPLAN Not.*, vol. 53, no. 2, Mar. 2018.
- [6] A. Pedram, A. Gerstlauer, and R. A. v. d. Geijn, “A high-performance, low-power linear algebra core,” in *ASAP 2011 - 22nd IEEE International Conference on Application-specific Systems, Architectures and Processors*, Sep. 2011, pp. 35–42.
- [7] T. J. Ham, L. Wu, N. Sundaram, N. Satish, and M. Martonosi, “Graphicionado: A high-performance and energy-efficient accelerator for graph analytics,” in *The 49th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-49, Piscataway, NJ, USA, 2016.
- [8] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, “EIE: efficient inference engine on compressed deep neural network,” *SIGARCH Comput. Archit. News*, vol. 44, no. 3, Jun. 2016.
- [9] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks,” in *Proceedings of the 43rd International Symposium on Computer Architecture*, ser. ISCA ’16, Piscataway, NJ, USA, 2016.
- [10] A. M. Caulfield *et al.*, “A cloud-scale acceleration architecture,” in *The 49th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-49, Piscataway, NJ, USA, 2016.
- [11] A. Putnam *et al.*, “A reconfigurable fabric for accelerating large-scale datacenter services,” in *Proceeding of the 41st Annual International Symposium on Computer Architecture*, ser. ISCA ’14, Piscataway, NJ, USA, 2014.
- [12] P. Francisco, “IBM PureData system for analytics architecture,” *IBM Redbooks*, pp. 1–16, 2014.
- [13] “Swarm 64,” <https://swarm64.com/>.
- [14] J. Fowers *et al.*, “A configurable cloud-scale DNN processor for real-time AI,” in *Proceedings of the 45th Annual International Symposium on Computer Architecture*, ser. ISCA ’18, Piscataway, NJ, USA, 2018.
- [15] I. Kuon, R. Tessier, and J. Rose, “FPGA architecture: Survey and challenges,” *Found. Trends Electron. Des. Autom.*, vol. 2, no. 2, Feb. 2008.
- [16] H. K.-H. So and C. Liu, *FPGA Overlays*. Cham: Springer International Publishing, 2016, pp. 285–305.
- [17] D. Koeplinger *et al.*, “Spatial: A language and compiler for application accelerators,” in *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2018. New York, NY, USA: ACM, 2018, pp. 296–311.
- [18] “Timescale,” <https://www.timescale.com/>.
- [19] “PostGIS,” <https://postgis.net/>.
- [20] “Oracle Spatial and Graph,” <https://www.oracle.com/database/technologies/spatialandgraph.html>.
- [21] A. Ratner, S. H. Bach, H. R. Ehrenberg, J. A. Fries, S. Wu, and C. Ré, “Snorkel: Rapid training data creation with weak supervision,” *CoRR*, vol. abs/1711.10160, 2017.
- [22] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, “Plasticine: A reconfigurable architecture for parallel patterns,” *SIGARCH Comput. Archit. News*, vol. 45, no. 2, pp. 389–402, Jun. 2017.
- [23] Y. Zhang, A. Rucker, M. Vilim, R. Prabhakar, W. Hwang, and K. Olukotun, “Scalable interconnects for reconfigurable spatial architectures,” in *Proceedings of the 46th International Symposium on Computer Architecture*, ser. ISCA ’19. New York, NY, USA: ACM, 2019, pp. 615–628.
- [24] T. H. Davenport, P. Barth, and R. Bean, “How big data is different,” *MIT Sloan Management Review*, vol. 54, no. 1, Fall 2012.
- [25] L. Dallemule and T. H. Davenport, “What’s your data strategy,” *Harvard Business Review*, vol. 95, no. 3, pp. 112–121, 2017.
- [26] E. F. Codd, “A relational model of data for large shared data banks,” *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.
- [27] J. Melton, “SQL language summary,” *ACM Comput. Surv.*, vol. 28, no. 1, pp. 141–143, Mar. 1996.
- [28] M. Zaharia *et al.*, “Apache Spark: a unified engine for big data processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [29] A. McAfee, E. Brynjolfsson, T. H. Davenport, D. Patil, and D. Barton, “Big data: the management revolution,” *Harvard business review*, vol. 90, no. 10, pp. 60–68, 2012.
- [30] Google. Introduction to BigQuery ML. [Online]. Available: <https://cloud.google.com/bigquery-ml/docs/bigqueryml-intro>
- [31] J. M. Hellerstein *et al.*, “The MADlib analytics library: Or MAD skills, the SQL,” *Proc. VLDB Endow.*, vol. 5, no. 12, pp. 1700–1711, Aug. 2012.
- [32] J. Dean *et al.*, “Large scale distributed deep networks,” in *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1*, ser. NIPS ’12. USA: Curran Associates Inc., 2012, pp. 1223–1231.
- [33] D. Abadi *et al.*, “The beckman report on database research,” *Commun. ACM*, vol. 59, no. 2, pp. 92–99, Jan. 2016.
- [34] Y. Wang, P. Li, and J. Cong, “Theory and algorithm for generalized memory partitioning in high-level synthesis,” in *Proceedings of the 2014 ACM/SIGDA International Symposium on Field-programmable Gate Arrays*, ser. FPGA ’14. New York, NY, USA: ACM, 2014, pp. 199–208.
- [35] J. Casper and K. Olukotun, “Hardware acceleration of database operations,” in *Proceedings of the 2014 ACM/SIGDA International Symposium on Field-programmable Gate Arrays*, ser. FPGA ’14. New York, NY, USA: ACM, 2014, pp. 151–160.
- [36] “OpenCAPI: a data-centric approach to server design,” <https://opencapi.org/wp-content/uploads/2016/09/OpenCAPI-Exhibit-SC17.pdf>.
- [37] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzyniak, and K. Asanović, “Chisel: Constructing hardware in a scala embedded language,” in *Proceedings of the 49th Annual Design Automation Conference*, ser. DAC ’12. New York, NY, USA: ACM, 2012, pp. 1216–1225.
- [38] “Design compiler: RTL synthesis,” <https://www.synopsys.com/support/training/rtl-synthesis/design-compiler-rtl-synthesis.html>.
- [39] K. Bhanushali and W. R. Davis, “FreePDK15: an open-source predictive process design kit for 15nm FinFET technology,” in *Proceedings of the 2015 Symposium on International Symposium on Physical Design*, ser. ISPD ’15. New York, NY, USA: Association for Computing Machinery, 2015, p. 165–170.

- [40] Y. Kim, W. Yang, and O. Mutlu, “Ramulator: A fast and extensible DRAM simulator,” *IEEE Comput. Archit. Lett.*, vol. 15, no. 1, pp. 45–49, Jan. 2016.
- [41] “Alveo U280 data center accelerator card,” <https://www.xilinx.com/products/boards-and-kits/alveo/u280.html>, 2019.
- [42] “Database and data analytics,” <https://www.xilinx.com/applications/data-center/database-data-analytics.html>.
- [43] D. Sidler, Z. Istvan, M. Owaida, K. Kara, and G. Alonso, “doppioDB,” in *Proceedings of the 2017 ACM International Conference on Management of Data - SIGMOD '17*. ACM Press, 2017.
- [44] M. Owaida, D. Sidler, K. Kara, and G. Alonso, “Centaur: A framework for hybrid CPU-FPGA databases,” in *2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, Apr. 2017.
- [45] E. S. Chung, J. D. Davis, and J. Lee, “LINQits: big data on little clients,” in *Proceedings of the 40th Annual International Symposium on Computer Architecture*, ser. ISCA '13, New York, NY, USA, 2013.
- [46] V. Dadu, J. Weng, S. Liu, and T. Nowatzki, “Towards general purpose acceleration by exploiting common data-dependence forms,” in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52, New York, NY, USA, 2019.
- [47] D. Mahajan, J. K. Kim, J. Sacks, A. Ardalan, A. Kumar, and H. Esmaeilzadeh, “In-RDBMS hardware acceleration of advanced analytics,” *Proc. VLDB Endow.*, vol. 11, no. 11, Jul. 2018.
- [48] P. Greenplum. About the Greenplum architecture. [Online]. Available: https://gpdb.docs.pivotal.io/5210/admin_guide/intro/arch_overview.html